

Multitarea y control de procesos

1. Concepto de proceso y de cambio de contexto

Linux, como sistema operativo multitarea, permite estar ejecutando de forma simultánea más de una tarea (proceso). Esto es así incluso aunque nuestro ordenador tenga únicamente una CPU, puesto que utiliza la técnica denominada **tiempo compartido** para que los diferentes procesos que se estén ejecutando compartan el tiempo de utilización de la CPU: durante un rato se ejecuta un proceso, durante otro rato se ejecuta otro proceso,...

Para hacer este reparto en el uso de la CPU, una parte del sistema operativo está dedicada a hacer una planificación de la misma utilizando para ello algún **algoritmo de planificación**.

Estamos hablando de procesos y no de programas porque son conceptos diferentes:

- ➡ Un **programa** es el archivo o conjunto de archivos que forman ese programa y que estarán almacenados en un soporte físico de memoria secundaria.
- ➡ En cambio, un **proceso** o **tarea** es una instancia de un programa en ejecución. Se habla de instancia porque un mismo programa puede generar más de un proceso.

Ejemplo:

Podemos tener iniciadas 2 sesiones de usuario en shells diferentes (uno en modo gráfico y otro en modo texto en un ttys (Control + Alt +F[12345...]) . En cada una de estas sesiones podemos estar ejecutando editor de texto (nano, vim...) pero de forma independiente, de tal manera que en una de las sesiones podemos estar creando un archivo de texto y en la otra podríamos estar modificando un archivo de configuración. De esta manera, tenemos 2 procesos ejecutándose que han sido generados por el mismo programa (el editor nano o **vi**).

Al planificar la CPU para repartir su uso entre varios procesos, hay que hacer un cambio de ejecución de un proceso a otro y esta tarea es más compleja de lo que pueda parecer. Esto es debido a que un proceso cuando se está ejecutando mantiene a la CPU y al resto del sistema en un determinado estado de ejecución, que seguramente no tendrá nada que ver con el estado de ejecución que necesita el siguiente proceso que está esperando que se le otorgue el uso de la CPU. Por lo tanto, cada vez que se realiza una conmutación entre procesos, es necesario guardar el estado del sistema tal como lo ha dejado el proceso que va a “salir” de la CPU para luego poder volver a recuperar ese estado cuando ese proceso vuelva a tener el control de la CPU y así continuar su ejecución en el punto en el que la había dejado. A este mecanismo de guardar el estado del sistema tal y como lo deja el proceso que sale de la CPU, y de recuperar el estado del sistema tal y como lo dejó el proceso que va a “entrar” en la CPU, se le denomina **cambio de contexto**.

2. Proceso padre y proceso hijo

Un proceso siempre es creado por otro proceso: al que crea un proceso se le denomina **proceso padre**, mientras que el proceso creado se denomina **proceso hijo**.

El primer proceso que se crea en el sistema es el proceso denominado **init** y él no tiene un proceso padre. Este proceso se encarga de crear otros muchos procesos como pueden ser las terminales virtuales (que en realidad son también procesos). De esta manera, un proceso que ejecutemos desde una terminal virtual será hijo del shell que se esté ejecutando en esa terminal virtual (normalmente del intérprete bash), que a su vez es hijo de la terminal virtual y ésta es hija del proceso **init**.

Cuando ejecutamos un programa desde la línea de comandos, es el shell quien maneja su ejecución: si la orden es un comando interno (**pwd**, **cd**, etc.), se ejecuta internamente sin generar nuevos procesos; pero si la orden no es interna (como el resto de programas y los comandos externos), entonces el shell crea un proceso hijo que ejecuta esa orden mientras el proceso padre (el shell) espera (duerme) hasta que el hijo termina su ejecución, momento en que el shell “se despierta” para interpretar la siguiente orden.

3. PID de un proceso: orden **ps**

Todo proceso está identificado unívocamente mediante un número. Dicho número se denomina **identificador de proceso** o, más comúnmente, **PID** (Process IDentifier). Este número se puede averiguar para así poder realizar ciertas operaciones sobre un proceso.

Hay varios comandos para conocer qué procesos se están ejecutando ahora mismo en el sistema y también obtener el PID de cada uno de ellos. El más usual de todos es el comando **ps**, cuya sintaxis es la siguiente:

ps [opciones]

Ejecutado sin opciones, este comando únicamente muestra los procesos que está ejecutando el usuario en su sesión en una determinada terminal virtual. La salida que presenta es similar a la siguiente:

PID	TTY	TIME	CMD
2798	tty2	00:00:00	bash
2906	tty2	00:00:00	ps

En este caso está mostrando una línea con unos títulos y 2 líneas con información de 2 procesos que son los que se están ejecutando en la sesión del usuario actual. El shell creó el proceso **ps** cuando el usuario se lo indicó por teclado y mientras se ejecutó el comando **ps**, el bash se quedó en estado de espera (durmiendo).

El significado de cada uno de los campos que aparecen es el siguiente:

- ➡ La columna **PID** indica el PID de cada proceso.
- ➡ La columna **TTY** muestra la terminal desde la que se lanzó la ejecución del proceso (en este caso, la consola es la 2). Hay procesos que no están asociados a ninguna terminal sino que son ejecutados por el sistema de forma automática y en este caso en esta columna aparece el símbolo ?
- ➡ La columna **TIME** indica el tiempo de CPU que utilizó el proceso, expresado en horas, minutos y segundos.
- ➡ La columna **CMD** nos muestra el nombre del proceso.

La orden **ps** tiene un montón de opciones y muchas de ellas, a diferencia de lo que ocurre con el resto de comandos de Linux, no van precedidas de un guión. Algunas opciones interesantes son las siguientes:

OPCIONES DE ps	SIGNIFICADO
x	Muestra, además de de los procesos generados en la propia terminal, aquellos que no son controlados por ninguna terminal.
a	Muestra todos los procesos generados en la terminal, incluso los de otros usuarios.
u	Muestra el listado con más opciones.
-e	Muestra todos los procesos presentes en el sistema.
-f	Mustra información detallada de cada proceso.
U nombre_usuario	Muestra los procesos que ha iniciado el usuario indicado.
t número_terminal	Muestra los procesos generados en la terminal indicada.

4. Ejecución en primer plano y en segundo plano

En muchas ocasiones los usuarios sólo ejecutan un proceso a la vez, que será aquél que han invocado desde el shell. Sin embargo, como ya sabemos, en Linux podemos ejecutar diferentes tareas al mismo tiempo, cambiando entre ellas según lo necesitemos.

Hay 2 formas de ejecutar un proceso:

1. En **primer plano** (foreground).
2. En **segundo plano** (background).

1. Ejecución de procesos en primer plano

Ésta es la ejecución que hemos hecho hasta ahora. De esta manera, cuando un proceso se ejecuta en primer plano, el shell no nos devuelve el control de la línea de órdenes hasta que ese proceso ha terminado de ejecutarse.

En la misma sesión, **no puede haber más de un proceso ejecutándose en primer plano**.

Este método de ejecución es el ideal para los procesos que requieran interactuar con el usuario, recibiendo de dicho usuario entradas por teclado y enviando al monitor las salidas que producen.

Ejemplo:

Procesos como el **nano** o el **vim**, son ejemplos claros para ser ejecutados en primer plano, ya que requieren interactuar de forma continua con el usuario.

2. Ejecución de procesos en segundo plano

Hay procesos que necesitan bastante tiempo para ejecutarse y que, además, durante su ejecución no muestran sus resultados en pantalla y no requieren ningún tipo de interacción con el usuario. Esta situación se produce cuando compilamos programas grandes, o también cuando queremos comprimir un montón de archivos grandes.

Si ejecutamos este tipo de procesos en primer plano, tenemos que esperar a que finalicen para poder realizar otra tarea puesto que el sistema no nos devuelve el control hasta que termine su ejecución.

Esta clase de procesos es ideal para ejecutar en **segundo plano** ya que un proceso ejecutado en este modo, se está ejecutando internamente y el sistema me devuelve el control de la línea de órdenes para que yo ejecute otro proceso (bien en primer plano, bien en segundo plano).

Por lo tanto, a diferencia del caso anterior, **en segundo plano podemos ejecutar más de un proceso de forma simultánea**.

¿Cómo se le dice al sistema que ejecute un proceso en segundo plano? Utilizando el operador **&** al final del comando.

Ejemplo:

Supongamos que queremos realizar una tarea que tiene una larga duración. Para ejemplificar crearemos un archivo de números aleatorios de tamaño 10GB con el comando **dd**.

Podríamos hacer esa búsqueda con el comando **find**:

1. Ejecución en primer plano:

```
dd if=/dev/urandom of=/tmp/Archivo1G.bin bs=1M count=1000
```

```
ls -lh /tmp/Arqui*
```

En este caso, no puedes ejecutar el **ls** hasta que no finaliza el **dd**.

2. Ejecución en segundo plano:

```
dd if=/dev/urandom of=/tmp/Archivo1G.bin bs=1M count=10000 &
```

```
ls -lh /tmp/Arqui*
```

En este segundo caso, el sistema me devuelve el control mientras ejecuta internamente el comando **dd**. Podríamos hacer más tareas a la vez que genera el archivo, como por ejemplo el **ls**. Verás que si ejecutas varias veces el comando **ls** el tamaño del archivo va creciendo poco a poco.

Cuando se ejecuta un proceso en segundo plano, el shell muestra 2 valores numéricos y nos da el control de la línea de órdenes. Esos 2 valores son:

- El PID del proceso.
- El número de tarea entre corchetes. El shell asigna un número consecutivo (comenzando en 1) a cada tarea que se está ejecutando.

Ambos valores pueden ser utilizados para referirse a esa tarea y realizar sobre ella ciertas acciones.

5. Gestión de procesos

A la hora de gestionar los procesos, el sistema pone a disposición del usuario varios comandos, operadores y combinaciones de teclas:

- Orden ps: ya hemos visto que muestra los procesos presentes en el sistema.
- Operador &: al final de un comando o programa, lo ejecuta directamente en segundo plano.
- **CTRL+C**: esta combinación de teclas sirve para intentar terminar la ejecución de un proceso **que se esté ejecutando en primer plano**. No todos los procesos responden a esta señal por lo que algunos no se terminarán usando esta combinación de teclas (por ejemplo, *man ls* no responde a esta señal).
- **CTRL+Z**: detiene la ejecución de un proceso **que se esté ejecutando en primer plano**. Detener un proceso no es lo mismo que terminarlo. Al detenerlo, el shell me devuelve el control de la línea de órdenes para que yo pueda realizar otra tarea, pero el proceso detenido sigue estando presente en el sistema y ocupando memoria. Luego, se puede recuperar la ejecución de ese proceso y, además, se haría exactamente en el punto en el que estaba cuando se detuvo.

Cuando un proceso es detenido, el sistema nos muestra un mensaje similar al siguiente:

[nº_tarea]+ Stopped nombre_proceso

donde **nº_tarea** es el número que el sistema le asigna a esa tarea y **nombre_proceso** es el nombre del proceso que se ha detenido.

- Orden jobs: es una orden interna que nos muestra los trabajos o tareas que están detenidos y también aquellos que están ejecutándose en segundo plano. La sintaxis de este comando es:

jobs [opciones]

Ejecutada sin opciones, esta orden nos muestra 3 columnas de información para cada tarea. El formato de la salida de este comando sería similar a la siguiente:

[nº_tarea] estado nombre_proceso

A la derecha de **[nº_tarea]** puede aparecer también el carácter **+** o también el carácter **-**. El **+** indica que éste es el primer proceso de la lista de procesos, mientras que el **-** indica que sería el siguiente de la lista. ¿Para qué sirve esto? El comando **fg** a secas pasaría a primer plano aquella tarea que tiene el símbolo **+**, mientras que el comando **bg** a secas pasaría a segundo plano aquella tarea que tiene el símbolo **+**.

La opción **-l** del comando **jobs** muestra también el PID de cada tarea.

Es importante entender que los números de tareas (**no los PID**) son independientes de cada shell. Esto quiere decir que un mismo usuario puede estar conectado en 2 terminales diferentes y tener un trabajo en una terminal que tenga el mismo número de tarea de otro trabajo que esté ejecutando en la otra terminal y sin embargo ambas tareas son diferentes.

Los procesos sí que son globales a todo el sistema con independencia de la terminal por lo que no puede haber 2 procesos con el mismo PID.

- Orden fg: esta orden también es interna y permite pasar a primer plano un proceso que está detenido, o bien uno que esté ejecutándose en segundo plano. La sintaxis de esta orden es:

fg %nº_tarea
o
fg nº_tarea

donde ***nº_tarea*** podemos obtenerlo con el comando ***jobs***.

▪ **Orden *bg***: esta orden también es interna y permite pasar a segundo plano un proceso que está detenido. La sintaxis de esta orden es:

bg %nº_tarea
 o
bg nº_tarea

donde ***nº_tarea*** podemos obtenerlo con el comando ***jobs***.

▪ **Orden *kill***: éste es un comando externo. Sirve para enviar una señal a un proceso. Normalmente se utiliza para terminar (matar) un proceso.

Para enviar una señal a un proceso, se puede hacer referencia al mismo o bien por su PID o bien por su número de tarea.

La sintaxis de esta orden es la siguiente:

kill [opciones] PID ...
kill [opciones] %nº_tarea ...

Usado sin opciones, envía la señal denominada SIGTERM al proceso indicado. De esta manera, se le está indicando al proceso que termine su ejecución por sí mismo.

A veces, un proceso no responde a esta señal y sin embargo queremos terminarlo a toda costa. Éste puede ser el caso de un proceso que se haya bloqueado. Para estos casos, hay que enviar a ese proceso la señal denominada SIGKILL para abortarlo inmediatamente con lo cual sería el sistema operativo el que abortase su ejecución al momento. La sintaxis para hacer esto sería:

kill -9 PID ...
kill -9 %nº_tarea ...

Otras opciones de interés son -s SIGCONT, -s SIGSTOP.

▪ **Orden *killall***: esta orden es similar a la orden ***kill*** pero en este caso podemos “matar” un proceso por su nombre, en lugar de por su PID. En este caso, enviaría una señal a todos los procesos que estén ejecutando el comando especificado.